Final Project Report

SecureWatch: Cybersecurity Event Monitoring System Using Apache CouchDB

Rahul Akula
Bhargava Karthik Boddeda
Sandeep Kumar Chavan
Vamshi Padakanti

Project Title

SecureWatch: Cybersecurity Event Monitoring System Using Apache CouchDB

Group Members

- Rahul Akula
- Bhargava Karthik Boddeda
- Sandeep Kumar Chavan
- Vamshi Padakanti

Database in Use

The project uses **Apache CouchDB**, a document-oriented NoSQL database. CouchDB is deployed using Docker and communicates with a Python/Flask web application through its HTTP API.

Project Summary

Cybersecurity systems continuously generate events such as failed login attempts, malware detections, phishing reports, and unusual network activity. These events need to be recorded and reviewed so that an analyst can identify suspicious behavior and track the status of an investigation.

The goal of *SecureWatch* is to build a simple cybersecurity event management system around this idea. A user can record security events, view existing incidents, filter them by type, severity, or status, and update an incident as it moves from *open* to *investigating* or

resolved. The application also provides a dashboard that summarizes the current state of the recorded incidents.

Security events are stored in Apache CouchDB as JSON documents and are accessed through a Flask backend.

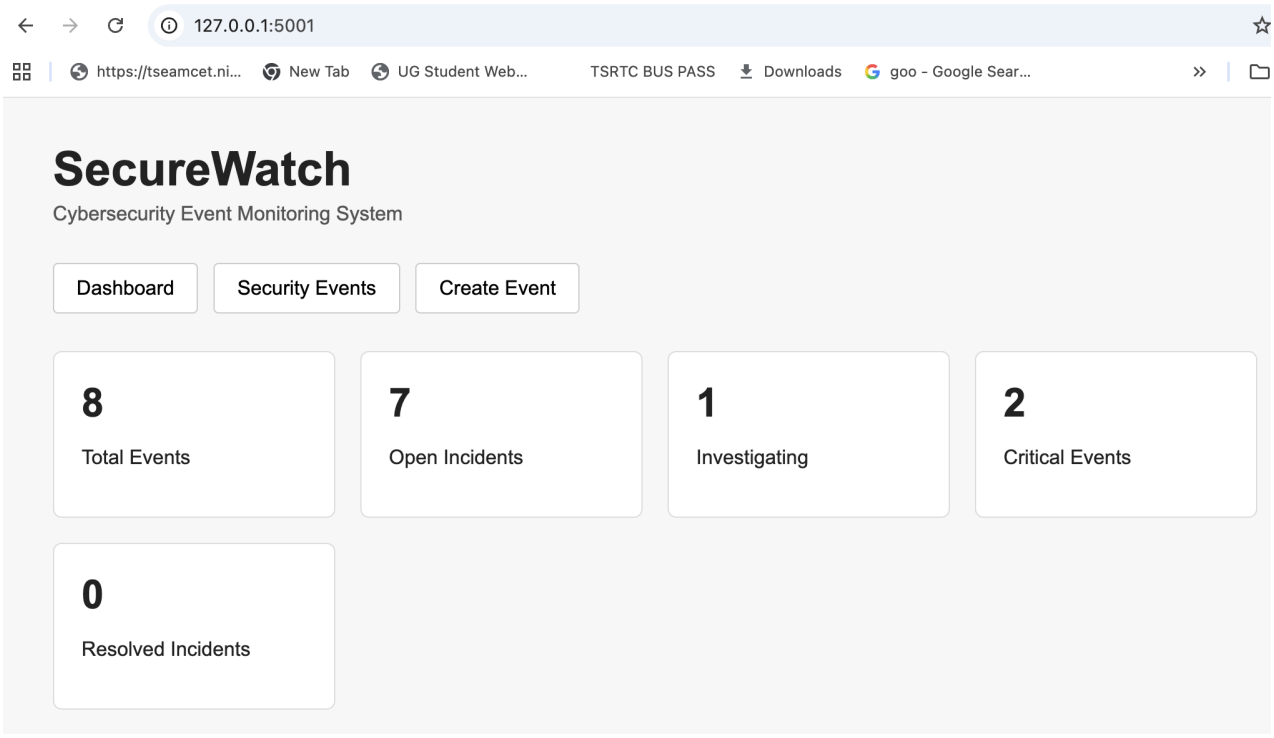


Figure 1: SecureWatch dashboard showing the current number of total, open, investigating, critical, and resolved cybersecurity events.

NoSQL Use Case

One of the main reasons for using a NoSQL database is that different cybersecurity events do not necessarily contain the same information.

For example, a failed-login event may contain:

- username,
- source IP address,
- hostname,
- number of failed attempts.

A malware event, however, may contain:

- hostname,
- malware name,

- source IP address,
- file path.

Similarly, phishing and network events contain their own event-specific attributes.

CouchDB is well suited for this application because it stores information as flexible JSON documents. Therefore, different event types can contain different attributes while still being stored in the same database. This allows SecureWatch to represent heterogeneous cybersecurity data without forcing every event into an identical relational structure.

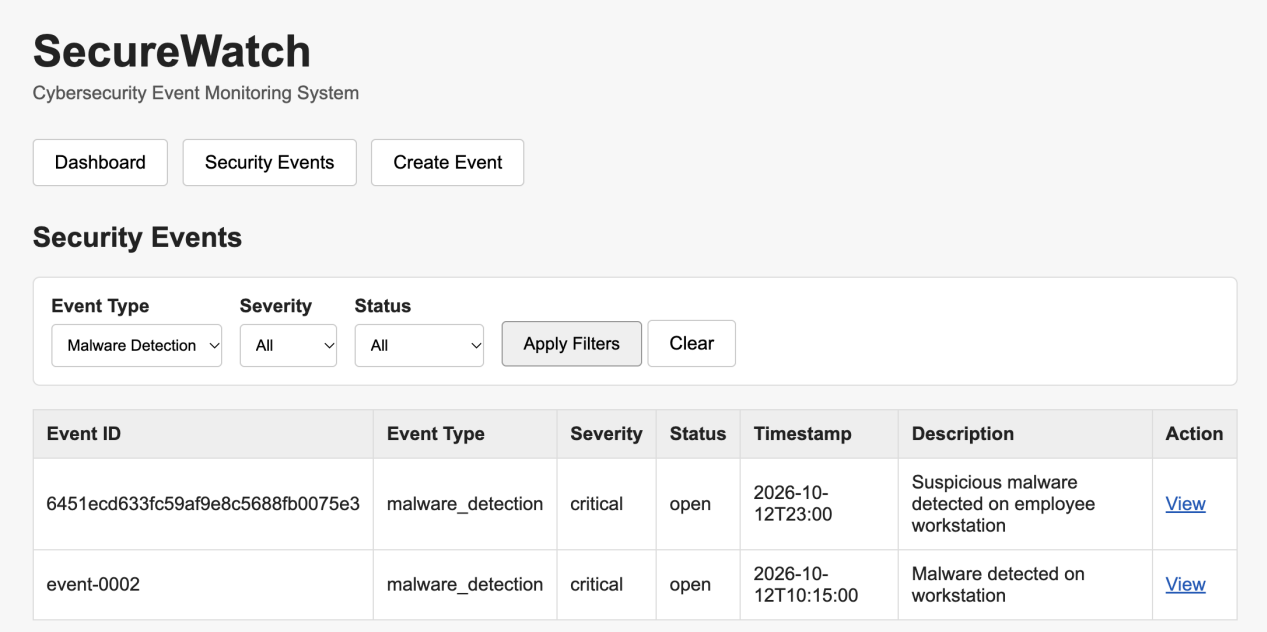


Figure 2: Security-event page demonstrating filtering by event type, severity, and status.

In addition to flexible document storage, the project demonstrates several CouchDB features:

- creation, retrieval, update, and deletion of JSON documents;
- Mango queries for filtering security events;
- JSON indexes for query optimization;
- document revision tracking using `_rev`;
- database replication.

Workflow and Data Flow

A typical SecureWatch workflow begins when a user creates a cybersecurity event through the web interface. The Flask application receives the submitted data and stores it as a JSON document in CouchDB. Stored events can then be retrieved, filtered using Mango queries, reviewed, and updated as an investigation progresses.

The overall application data flow is:

$$UserInterface \rightarrow FlaskApplication \rightarrow ApacheCouchDB$$

Results returned by CouchDB are processed by Flask and displayed back to the user through the SecureWatch interface. At the application level, the typical workflow is:

$$Create \rightarrow Review \rightarrow Filter \rightarrow Update \rightarrow Resolve$$

CouchDB replication is additionally used to copy the primary `securewatch` database to `securewatch_backup`.

Project Status

The SecureWatch project has been successfully completed. The final implementation includes the web application, CouchDB database integration, database operations, querying, replication, and project documentation.

The completed system provides the following functionality:

- Deployment of Apache CouchDB using Docker.
- Integration of CouchDB with a Python/Flask web application.
- Storage of different cybersecurity event types as flexible JSON documents.
- Create, read, update, and delete (CRUD) operations.
- Event filtering using CouchDB Mango queries.
- JSON indexing for query optimization.
- CouchDB document revision handling using `_rev`.
- Dashboard displaying live cybersecurity event statistics.
- Database replication from `securewatch` to `securewatch_backup`.
- Complete project source code and re-creation documentation hosted on GitHub.

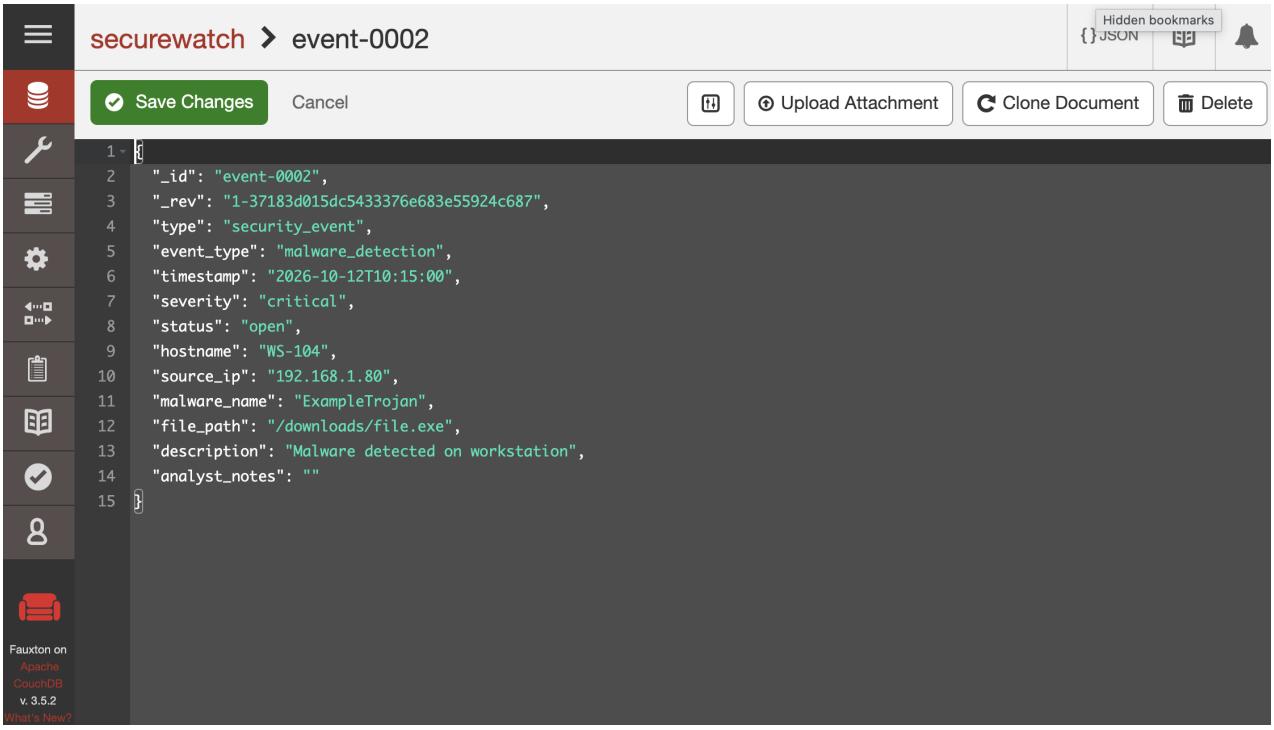


Figure 3: Example CouchDB JSON document for a malware-detection event, showing event-specific attributes stored in the `securewatch` database.

The filtering operation is performed using CouchDB Mango queries rather than filtering only in the browser. The Flask application constructs a query based on the selected filters and retrieves the matching documents from CouchDB.

Database Replication

A one-time replication of the main `securewatch` database has also been performed to a second database named `securewatch_backup`. This demonstrates CouchDB's built-in replication capability and provides a simple example of maintaining a second copy of the database.

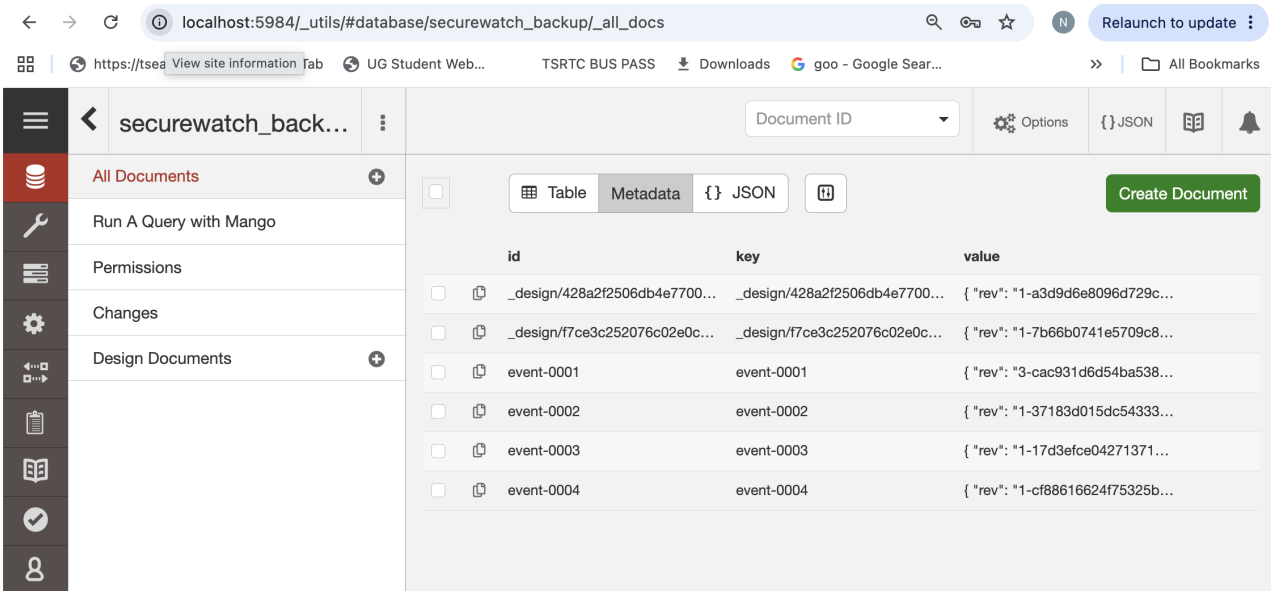


Figure 4: Replication of the `securewatch` database to `securewatch_backup`.

Technologies Used

- **Database:** Apache CouchDB
- **Backend:** Python and Flask
- **Frontend:** HTML, CSS, and JavaScript
- **Database Deployment:** Docker
- **Database Administration:** CouchDB Fauxton
- **Version Control:** Git and GitHub

Member Roles

- **Rahul Akula:** CouchDB setup and configuration, database design, implementation of NoSQL operations, and CouchDB–Flask integration.
- **Bhargava Karthik Boddeda:** Flask backend development, application logic, and implementation of create, update, delete, and query functionality.
- **Sandeep Kumar Chavan:** Front-end development, user-interface design, and integration of the dashboard, event forms, and event-management pages.
- **Vamshi Padakanti:** Application testing, project documentation, GitHub re-creation instructions, and preparation of the final demonstration and presentation.

GitHub Repository

The source code and project re-creation instructions are maintained in the following GitHub repository:

`https://github.com/RahulAkula17/SecureWatch`

The repository contains the Flask application, HTML templates, CSS files, Docker configuration, Python dependencies, and the instructions required to recreate and run the project.

Conclusion

SecureWatch demonstrates the practical use of Apache CouchDB in a cybersecurity-oriented application. The project combines flexible JSON document storage, CouchDB database operations, Mango queries, revision tracking, and replication with a Flask-based web interface. The completed system provides a functional example of how a document-oriented NoSQL database can be used to manage heterogeneous cybersecurity event data.